

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hafta 1'den taşıdıklarınız: referans (int& — takma ad), const, namespace, .hpp/.cpp + CMake iskeleti ve geometri::Nokta değer tipi. C'den tanıdık gelen: malloc/free, işaretçi (*/&) ve fonksiyon tanımı/çağrısı (yalnız karşılaştırma noktası). Bu hafta aynı fonksiyon ve referans temelini aşırı yükleme, dört parametre geçme biçimi ve new/delete bellek modeliyle derinleştirirsiniz.

» Aşırı Yükleme & Varsayılan Parametre

Aşırı Yükleme (Overloading)

Aynı ad, farklı **imza** (parametre sayısı/tipi). Derleyici doğru sürümü çağrı anındaki argümanlara bakarak derleme zamanında seçer. Dönüş tipi imzanın parçası DEĞİLDİR — C'de karşılığı yoktur (her ad benzersiz olmalı).

```
1 int alan(int kenar); // kare
2 int alan(int taban, int yukseklik); // dikdortgen
3 double alan(double yaricap); // daire
4 alan(4); alan(3,5); alan(2.0); // derleyici secer
```

Varsayılan Parametre

tip ad = deger — çağıran atlarsa bildirimdeki değer kullanılır. Varsayılanlı parametreler listenin sonunda olmalı.

```
1 int guc(int taban, int us = 2);
2 guc(3); // us=2 -> 9
3 guc(3, 3); // us=3 -> 27
```

» Parametre Geçme: Karar Tablosu

T param (değer)	Kopya oluşur; çağırın değişmez
T& param (referans)	Kopyasız takma ad; çağırın değişir
const T& param	Kopyasız + değiştirilemez — büyük tipte VARSAYILAN
T* param (işaretçi)	Adres; nullptr olabilir, yeniden atanabilir

Ne Zaman Hangisi?

- Küçük tip (int, double, Nokta) → değere göre.
- Büyük tip, değişecek → T&.
- Büyük tip, yalnız okunacak → const T& (genel varsayılan).
- Parametre isteğe bağlı/boş olabilir → T* (tek durum — referans bunu karşılayamaz).

```
1 void oteleDeger(Nokta n); // kopya, degismez
2 void oteleReferans(Nokta& n); // takma ad, degisir
3 void oteleConstReferans(const Nokta& n); // kopyasiz, salt okunur
```

```
4 void oteleIsaretci(Nokta* n) {
5     if (n == nullptr) return; // her zaman once kontrol
6     n->x += 10.0;
7 }
8 oteleIsaretci(&nokta); // caginda & alinir
```

» Yığın (Stack) vs Öbek (Heap)

Yığın — Otomatik Ömür

Fonksiyon/blok kapsamına giren yerel değişkenler otomatik ayrılır, kapsam bitince otomatik geri alınır. Hızlı, $O(1)$; ömrü programcının elinde değil (masaüstü: iş bitince kağıtlar kendiliğinden toplanır).

Öbek — Dinamik Ömür

new ile açıkça istenir, delete çağrılana kadar yaşar; kapsamdan bağımsız (kiralık depo: siz kiralamadan sizin olmaz, anahtarı iade etmeden boşalmaz). Ayırma maliyeti öbeğin durumuna bağlı, yığından pahalı.

```
1 Nokta yerel{5.0, 5.0}; // yigin, stackOrnegi() donunce silinir
```

```
2 Nokta* dinamik = new Nokta{7.0, 7.0}; // obek
3 delete dinamik; // acikca geri verilir
4 dinamik = nullptr; // iyi aliskanlik
```

```
5 Nokta* p = nullptr;
6 if (p == nullptr) { /* guvenli */ }
```

new T(...) / delete p;	Tekil nesne ayır/bırak
new T[n] / delete[] p;	Dizi ayır/bırak — tip eşleşmeli
malloc/free (C)	new/delete (C++) — tip güvenli, boyut elle değil

» Sık Yapılan Hatalar

Ortak Hata / Tuzak

- Bellek sızıntısı:** new ile ayrılan bellek karşılığı delete çağrılmadan işaretçi üzerine yazılır/kapsam dışına çıkar — bellek program bitene kadar geri alınamaz.
- Sarkan işaretçi:** delete p; sonrası p hâlâ eski adresi taşır; o adrese tekrar erişmek (p->x) tanımsız davranıştır. delete'ten hemen sonra p = nullptr; yazın.
- Tek bir atlanan kod yolu (erken return, unutilan dal) tek başına sızıntıya yeter — kaç satır

doğru olursa olsun biri eksikse sonuç aynıdır.

- Büyük tipi (std::string, uzun liste) hiç değiştirmeyecekken değere göre almak: gereksiz kopya — const T& tercih edin.

» RAII'ye İlk Bakış

📖 Tanım: RAII

Kaynağın edinimi nesnenin **oluşturulmasına**, bırakılması nesnenin **kapsam dışına çıktığı anda otomatik çalışan** özel bir fonksiyona bağlanır — otel oda kartı: check-in'de oda sizindir, süreniz dolunca otel kendiliğinden kapatır, anahtarı elle iade etmeyi hatırlamanız gerekmez.

araclar::IntTampon — Elle Sarılmış Örnek

```
1 struct IntTampon {
2     int* veri;
3     int boyut;
4     IntTampon(int n); // olusturuldunda cagrilir
5     ~IntTampon(); // kapsam sonunda otomatik cagrilir
6 };
7 // govdeler:
8 IntTampon::IntTampon(int n) { veri = new int[n]; boyut = n; }
9 IntTampon::~IntTampon() { delete[] veri; }
```

Elle Yönetim vs RAII

```
1 int* elleTampon = new int[5];
2 // ... doldur, kullan ...
3 delete[] elleTampon; // her yol bu satira ugramali
4
5 {
6     aracler::IntTampon tampon(5); // olusturuldu
7     // ... kullan ...
8 } // blok sonu -> yikici otomatik cagrilir, temizlendi
```

RAII new/delete çiftini ORTADAN KALDIRMAZ; tek bir yere, unutulamayacak biçimde (kurucu/yıkıcı) taşır — biçimsel tanım Hafta 3'te.

🔧 Kendinizi Sınavın 1

- Yalnızca dönüş tipiyle ayrılan iki fonksiyon neden derlenmez? *İmza dönüş tipini içermez; derleyici çağ-*

rıyı parametre listesine bakarak çözer.

- `new` ile ayrılan bir nesne hiç `delete` edilmezse ne olur? Bellek sızıntısı — o bellek program sonlanana kadar geri alınmaz.
- RAII elle `new/delete` riskini nasıl azaltır? Bırakma adımını nesnenin kapsam sonunda otomatik çalışan bir fonksiyona bağlar, elle hatırlamayı gerektirmez.

Özet

Bu hafta fonksiyonları (aşırı yükleme, varsayılan parametre) ve dört parametre geçme biçimini (değer/`T&/const T&/T*`) derinleştirdiniz; yığın/öbek ayrımını, `new/delete/ nullptr`'ı, sızıntı ve sarkan işaretçi tuzaklarını, RAII'nin ilk somut örneğini (`IntTampon`) gördünüz. **Hafta 3:** veriyi ve davranışı bir sınıf çatısı altında birleştireceksiniz — kapsülleme, kurucu/yıkıcı, `this`.

5 • İzleme **Nerede Kullanılır:** Bir fonksiyon imzası tasarlarken: küçük tipi (`int`, `double`) değere göre, büyük nesneyi salt okuyacaksınız `const &` ile, çağıranda değiştirecekseniz düz referansla (`T&`), boş olabilen/isteğe bağlı parametreyi işaretçiyle (`T*`) geçin. Kısa ömürlü yerel veriyi yığın-da tutun; ömrü kapsamı aşan veriyi `new` ile öbekten isteyip her kod yolunda `delete` ile geri verin, `delete` sonrası işaretçiyi `nullptr`'a çekin. Elle bırakmayı unutma riskini, kaynağı nesnenin ömrüne bağlayarak (kapsam sonunda otomatik temizlik) ortadan kaldırın.